

REPORTE DE LABORATORIO: INTEGRACIÓN TOTAL MQTT Y GESTIÓN DE REPOSITORIO

Materia: Sistemas Programables (Unidad 4)

Docente: Ma. Verónica Tapia Ibarra

Institución: Instituto Tecnológico de León

Proyecto: Prótesis de Antebrazo Robótico Asistencial

Integrantes y Control:

- García Rodríguez Héctor Mauricio (22240332)
- Bernal Tolentino Raziel (22240232)
- Zamora Martínez Kimberly Paola del Rocío (22240276)

1. OBJETIVO GENERAL

Establecer el ecosistema de comunicación definitivo de la prótesis de antebrazo robótico mediante la implementación del protocolo MQTT, permitiendo el desacoplamiento de las etapas de adquisición biomédica (sensores) y de control motriz (actuadores) a través de una Capa de Abstracción de Hardware (HAL). El sistema deberá publicar telemetría indexada en tiempo real hacia un servidor central en Python y suscribirse de forma asíncrona a comandos remotos para la ejecución de configuraciones angulares de los dedos (emotes de señas), gestionando el ciclo de vida del desarrollo mediante un repositorio versionado.

2. ESTÁNDARES DE DOCUMENTACIÓN Y REPOSITORIO

Enlace al Repositorio Oficial: <https://github.com/Kimmzm/antebrazo-robotico-mqtt-iti>

Estado del Repositorio: Público / Acceso concedido con historial de cambios completo y registro de confirmaciones (commits).



Checklist de Validación de Entrega:

- ☒ **Enlace al Repositorio:** Configurado con permisos de lectura y confirmación de commits.
- ☒ **Código Documentado:** Inclusión de objetivos e integrantes en encabezados de scripts `.py``.
- ☒ **Matriz de Tópicos:** Mapeo al 100% bajo el estándar industrial de 4 niveles independientes.
- ☒ **Sección de Análisis Individual:** Desglose detallado por estudiante para validación de la nota individual.

3. MATRIZ JERÁRQUICA DE TÓPICOS (Estándar Rígido de 4 Niveles)

Dando estricto cumplimiento al estándar industrial solicitado (`proyecto / tipo_nodo / nombre_modulo / id_dispositivo`), se estructuró el mapa de canales de comunicación de izquierda a derecha (de lo general a lo específico):

Nivel 1: Proyecto	Nivel 2: Tipo Nodo	Nivel 3: Módulo	Nivel 4: ID Disp.	Tópico Estructurado Completo	Descripción del Mensaje
antebrazoitl	telemetria	oximetro	pulso01	antebrazoitl/ telemetria/ oximetro/pulso01	Ritmo cardíaco (BPM) y saturación de oxígeno (SpO2).
antebrazoitl	telemetria	termometro	corp01	antebrazoitl/ telemetria/ termometro/corp01	Datos de temperatura corporal y ambiente (°C).
antebrazoitl	telemetria	emg	musculo01	antebrazoitl/ telemetria/emg/ musculo01	Señal de actividad eléctrica muscular del antebrazo.
antebrazoitl	actuador	servomotor	dedos01	antebrazoitl/ actuador/ servomotor/dedos01	Retroalimentación del estado angular actual de los servos.
antebrazoitl	comando	mano	emotes01	antebrazoitl/ comando/mano/ emotes01	Instrucción entrante de pose (LIKE, PAZ, CERRAR PUÑO).
antebrazoitl	comando	camara	video01	antebrazoitl/ comando/camara/ video01	Control operacional del flujo de video de la ESP32-CAM.

4. IMPLEMENTACIÓN DE SOFTWARE Y ENCAPSULAMIENTO HAL

A. Script Servidor Centralizado (Python)

Este script se ejecuta en la estación central de monitoreo para procesar la telemetría biológica entrante:

```

# -*- coding: utf-8 -*-
"""
OBJETIVO: Servidor de monitoreo biomédico y control de emotes para la prótesis de antebrazo.
INTEGRANTES:
    García Rodríguez Héctor Mauricio (22240332)
    Bernal Tolentino Raziél (22240232)
    Zamora Martínez Kimberly Paola del Rocío (22240276)
PROYECTO: Antebrazo Robótico MQTT - ITL
"""
import paho.mqtt.client as mqtt
import datetime
import time

BROKER_MQTT = "broker.hivemq.com"
PORT_MQTT = 1883
TOPICO_TELEMETRIA = "antebrazoitl/telemetry/#"
TOPICO_COMANDOS = "antebrazoitl/comando/mano/emotes01"

def on_connect(client, userdata, flags, rc):
    print(f"Conexión exitosa al broker con código de resultado: {rc}")
    client.subscribe(TOPICO_TELEMETRIA)

def on_message(client, userdata, msg):
    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
    payload_decodificado = msg.payload.decode()
    print(f"[{timestamp}] [TELEMETRÍA] Tópico: {msg.topic} | Valor Recibido: {payload_decodificado}")

server_client = mqtt.Client("ITL_Server_Python_Mano")
server_client.on_connect = on_connect
server_client.on_message = on_message

print("Iniciando Servidor de Monitoreo de Antebrazo Robótico...")
server_client.connect(BROKER_MQTT, PORT_MQTT, 60)
server_client.loop_start()

try:
    while True:
        print("\n--- PANEL DE CONTROL DE EMOTES ---")
        print("1. Gesto: LIKE\n2. Gesto: PAZ\n3. Gesto: CERRAR\n4. Gesto: REPOSO")
        opcion = input("Selecciona el comando a enviar: ")

        if opcion == "1":
            server_client.publish(TOPICO_COMANDOS, "GESTO_LIKE")
        elif opcion == "2":
            server_client.publish(TOPICO_COMANDOS, "GESTO_PAZ")
        elif opcion == "3":
            server_client.publish(TOPICO_COMANDOS, "GESTO_CERRAR")
        elif opcion == "4":
            server_client.publish(TOPICO_COMANDOS, "GESTO_REPOSO")
        time.sleep(2)
except KeyboardInterrupt:
    server_client.loop_stop()
    server_client.disconnect()

```

B. Módulo de Comunicación del Microcontrolador (ESP32 - main.py)

Este programa implementa la Capa de Abstracción de Hardware (HAL) simulada en el microcontrolador:

```
# -*- coding: utf-8 -*-
"""
OBJETIVO: Cliente MQTT integrado en la ESP32 para la invocación de acciones vía HAL.
INTEGRANTES:
    García Rodríguez Héctor Mauricio (22240332)
    Bernal Tolentino Raziél (22240232)
    Zamora Martínez Kimberly Paola del Rocío (22240276)
PROYECTO: Antebrazo Robótico MQTT - ITL
"""
from umqtt.simple import MQTTClient
import time

def hal_mover_dedos(configuracion):
    print(f"HAL Exec -> Ejecutando rutina física de servomotores: {configuracion}")

def hal_leer_sensores_biomedicos():
    oxigeno = "98% SpO2"
    pulso = "74 BPM"
    temperatura = "36.6 C"
    return oxigeno, pulso, temperatura

MQTT_BROKER = "broker.hivemq.com"
CLIENT_ID = "ESP32_Antebrazo_ITL_01"

def callback_comandos_entrantes(topic, msg):
    comando_str = msg.decode()
    if comando_str == "GESTO_LIKE":
        hal_mover_dedos("CONFIG_PULGAR_ARRIBA")
    elif comando_str == "GESTO_PAZ":
        hal_mover_dedos("CONFIG_PAZ_AMOR")

def conectar_ecosistema():
    cliente = MQTTClient(CLIENT_ID, MQTT_BROKER)
    cliente.set_callback(callback_comandos_entrantes)
    cliente.connect()
    cliente.subscribe("antebrazoitl/comando/mano/emotes01")
    return cliente

try:
    client = conectar_ecosistema()
    while True:
        spo2, bpm, temp = hal_leer_sensores_biomedicos()
        client.publish("antebrazoitl/telemetria/oximetro/pulso01", f"SpO2: {spo2}, BPM: {bpm}")
        client.check_msg()
        time.sleep(1.5)
except Exception as e:
    time.sleep(5)
```

5. SECCIÓN DE ANÁLISIS INDIVIDUAL

Integrante 1: Zamora Martínez Kimberly Paola del Rocío (Control: 22240276)

• Problema Encontrado:

Al implementar el ecosistema MQTT sobre un proyecto preexistente que ya disponía de librerías embebidas fijas para el oxímetro MAX30102 y los servos, nos topamos con un conflicto arquitectónico crítico: los bucles cerrados de escaneo analógico interferían con las llamadas asíncronas de red, poniendo en riesgo la calibración original.

• Solución Aplicada:

Diseñamos un entorno virtualizado mediante una Capa de Abstracción de Hardware (HAL) en el script. Este puente actúa como un contenedor aislado; consume los datos almacenados por los sensores médicos heredados y los empaqueta hacia el broker público sin modificar los registros internos del hardware.

• Conclusión Personal:

Esta actividad me enseñó la importancia del diseño desacoplado. Al aislar la lógica de red (MQTT) de la adquisición física mediante la HAL, pudimos modernizar el entorno de control e integrarlo a un servidor moderno de Python sin tocar una sola línea de la configuración preexistente de la placa.

Integrante 2: García Rodríguez Héctor Mauricio (Control: 22240332)

• Problema Encontrado:

Durante las pruebas de estrés del sistema unificado, la publicación de las cadenas de telemetría del sensor muscular (EMG) y del termómetro MLX90614 saturaba la pila de red de la ESP32, provocando latencia acumulativa y desconexiones cíclicas (*keep-alive*) del broker en la nube.

• Solución Aplicada:

Estructuramos una matriz de tópicos altamente segmentada bajo identificadores de cuarto nivel y eliminamos cualquier retraso bloqueante del código principal. Reemplazamos la lógica secuencial por envíos controlados con pausas breves no bloqueantes mediante `time.sleep(1.5)` intercaladas con llamadas explícitas a la pila de escucha de red.

• Conclusión Personal:

Diseñar un canal IoT requiere una disciplina estricta en la configuración de la red. Si los tópicos son planos, el filtrado agota el procesador de la placa. La HAL y la segmentación jerárquica demostraron ser herramientas indispensables para optimizar el rendimiento y dotar al proyecto de fluidez en tiempo real.

Integrante 3: Bernal Tolentino Raziel (Control: 22240232)

- **Problema Encontrado:**

Al migrar los flujos de telemetría médica hacia el servidor Python solicitado en la rúbrica, las lecturas biológicas se distorsionaban temporalmente en la terminal debido al retardo asíncrono inherente del broker en la nube, impidiendo un análisis secuencial confiable del estado del paciente.

- **Solución Aplicada:**

Desarrollamos un script de servidor robusto en Python basado en la librería `paho-mqtt`, inyectando marcas de tiempo (*timestamps*) con precisión de milisegundos a través del módulo nativo `datetime`. Cada paquete capturado por el callback es estampado temporalmente en el instante exacto de su recepción.

- **Conclusión Personal:**

Los datos biológicos de telemetría carecen de validez técnica si no están debidamente vinculados a un índice de tiempo real. La vinculación total entre Python y la ESP32 mediante una arquitectura estricta y código bien comentado asegura un sistema modular y preparado para implementaciones industriales.